부록 · Git · GitHub

Git + GitHub 초보자 명령어 100선

사용자 설정 · 파일 올리기 · 파일 관리 · 버전 관리 · Branch · GitHub

1. Git과 GitHub를 먼저 이해하기

Git은 내 PC에서 파일의 변경 이력과 버전을 관리하는 도구이고, GitHub는 Git 저장소를 인터넷에 보관·공유·협업하는 서비스임. Git의 user.name은 GitHub 로그인 아이디와 반드시 같은 값일 필요는 없으며 커밋 작성자 정보로 사용됨.

GitHub 명령줄 인증은 HTTPS 또는 SSH를 사용함. 초보자는 GitHub CLI를 설치한 뒤 gh auth login으로 로그인하면 편리함. HTTPS에서 GitHub 계정 비밀번호를 Git 인증 비밀번호로 직접 사용하는 방식 대신 GitHub CLI, 자격증명 관리자, Personal Access Token 등을 사용함.

2. 초보자가 가장 먼저 외울 작업 순서

순서	명령	뜻	언제 사용?
1	git status	현재 상태 확인	작업 시작 전/후
2	git add .	변경 파일을 커밋 준비	파일 수정 후
3	git commit -m "설명"	현재 상태를 버전으로 저장	작업 단위 완료 후
4	git pull	GitHub 최신 변경 받기	push 전 또는 작업 시작 전
5	git push	내 커밋을 GitHub로 업로드	커밋 후

3. 처음 프로젝트를 GitHub에 올리는 대표 예제

```bash
git config --global user.name "사용자이름"
git config --global user.email "이메일"

cd my-project
git init
git add .
git commit -m "프로젝트 최초 등록"
git branch -M main
git remote add origin https://github.com/OWNER/REPO.git
git push -u origin main
```

4. Git + GitHub 명령어 정확히 100개

#	분류	기능	명령어	초보자 설명
1	설치·확인	Git 버전 확인	git --version	Git이 설치되어 있는지와 버전을 확인함.
2	설치·확인	Git 도움말	git help	Git 전체 도움말을 엶.
3	설치·확인	특정 명령 도움말	git help commit	commit 명령의 자세한 설명을 확인함.

#	분류	기능	명령어	초보자 설명
4	사용자 설정	전역 사용자 이름 설정	git config --global user.name "Hong Gil Dong"	앞으로 만드는 커밋에 기록할 이름을 설정함.
5	사용자 설정	전역 이메일 설정	git config --global user.email "you@example.com"	커밋에 기록할 이메일을 설정함.
6	사용자 설정	이름 확인	git config --global user.name	설정한 사용자 이름을 확인함.
7	사용자 설정	이메일 확인	git config --global user.email	설정한 이메일을 확인함.
8	사용자 설정	전체 설정 확인	git config --global --list	전역 Git 설정을 한 번에 확인함.
9	사용자 설정	현재 저장소 이름 설정	git config user.name "Project User"	현재 프로젝트에서만 다른 이름을 사용함.
10	사용자 설정	현재 저장소 이메일 설정	git config user.email "project@example.com"	현재 프로젝트에서만 다른 이메일을 사용함.
11	사용자 설정	기본 브랜치 main 설정	git config --global init.defaultBranch main	새 저장소의 기본 브랜치를 main으로 설정함.
12	사용자 설정	기본 에디터 VS Code	git config --global core.editor "code --wait"	Git 편집기로 VS Code를 사용함.
13	GitHub 인증	GitHub CLI 로그인	gh auth login	브라우저/HTTPS/SSH 방식으로 GitHub 계정에 인증함.
14	GitHub 인증	로그인 상태 확인	gh auth status	현재 GitHub CLI 로그인 계정을 확인함.
15	GitHub 인증	GitHub CLI 로그아웃	gh auth logout	현재 GitHub CLI 인증을 해제함.
16	저장소	현재 폴더 Git 시작	git init	현재 폴더를 Git 저장소로 만듦.
17	저장소	새 폴더와 함께 시작	git init my-project	my-project 폴더를 만들고 Git 저장소로 초기화함.
18	저장소	GitHub 저장소 복제	git clone https://github.com/OWNER/REPO.git	GitHub 프로젝트를 PC로 내려받음.
19	저장소	SSH로 복제	git clone git@github.com:OWNER/REPO.git	SSH 인증을 사용해 저장소를 복제함.
20	저장소	다른 폴더명으로 복제	git clone https://github.com/OWNER/REPO.git myapp	원격 저장소를 myapp 폴더로 복제함.
21	상태	현재 상태 확인	git status	변경·추가·삭제된 파일과 스테이징 상태를 확인함.
22	상태	간단 상태 확인	git status -s	상태를 짧은 형식으로 확인함.
23	파일 추가	파일 1개 스테이징	git add README.md	README.md만 다음 커밋 대상으로 올림.
24	파일 추가	현재 폴더 전체 추가	git add .	현재 폴더 아래 변경 파일을 스테이징함.

#	분류	기능	명령어	초보자 설명
25	파일 추가	전체 변경 추가	git add -A	추가·수정·삭제된 모든 변경을 스테이징함.
26	파일 추가	특정 확장자 추가	git add '*.java'	Java 파일들을 스테이징함.
27	파일 추가	대화식 추가	git add -p	변경 부분을 확인하면서 선택적으로 스테이징함.
28	파일 관리	Git에서 파일 삭제	git rm old.txt	파일을 삭제하고 삭제 사실도 스테이징함.
29	파일 관리	폴더 재귀 삭제	git rm -r old-folder	폴더와 내부 파일을 Git에서 삭제함.
30	파일 관리	파일은 남기고 추적 해제	git rm --cached secret.txt	PC 파일은 유지하면서 Git 추적만 중단함.
31	파일 관리	폴더 추적 해제	git rm -r --cached build/	build 폴더는 남기고 Git 추적에서 제외함.
32	파일 관리	파일 이름 변경	git mv old.txt new.txt	Git이 추적하는 파일 이름을 변경함.
33	파일 관리	파일 이동	git mv app.txt docs/app.txt	추적 파일을 다른 폴더로 이동함.
34	파일 복구	작업 파일 복구	git restore README.md	커밋/인덱스 기준으로 README 변경을 되돌림.
35	파일 복구	전체 작업 파일 복구	git restore .	스테이징하지 않은 현재 폴더 변경을 되돌림.
36	파일 복구	스테이징 취소	git restore --staged README.md	add한 파일을 스테이징 영역에서 내림.
37	파일 복구	전체 스테이징 취소	git restore --staged .	스테이징한 전체 파일을 다시 작업 영역 변경으로 돌림.
38	차이 확인	작업 파일 차이	git diff	아직 스테이징하지 않은 변경 내용을 확인함.
39	차이 확인	스테이징 차이	git diff --staged	커밋 예정인 변경 내용을 확인함.
40	차이 확인	두 커밋 비교	git diff HEAD~1 HEAD	직전 커밋과 현재 커밋의 차이를 확인함.
41	커밋	커밋 생성	git commit -m "첫 번째 커밋"	스테이징된 변경을 하나의 버전으로 저장함.
42	커밋	추적 파일 add+commit	git commit -am "수정 내용"	이미 추적 중인 수정/삭제 파일을 바로 커밋함.
43	커밋	마지막 커밋 메시지 수정	git commit --amend -m "수정된 메시지"	가장 최근 커밋 메시지를 고침.
44	커밋	마지막 커밋에 파일 추가	git commit --amend --no-edit	누락 파일을 add한 뒤 기존 메시지를 유지하며 최근 커밋에 합침.
45	이력	커밋 로그	git log	커밋 기록을 자세히 확인함.
46	이력	한 줄 로그	git log --oneline	커밋 기록을 한 줄씩 간단히 봄.

#	분류	기능	명령어	초보자 설명
47	이력	그래프 로그	git log --oneline --graph --all	브랜치와 병합 흐름을 그래프로 확인함.
48	이력	최근 5개 로그	git log -5 --oneline	최근 5개 커밋만 확인함.
49	이력	파일 변경 이력	git log -- README.md	특정 파일과 관련된 커밋을 확인함.
50	이력	커밋 상세 보기	git show HEAD	현재 최신 커밋의 변경 내용을 확인함.
51	이력	특정 커밋 보기	git show abc1234	지정한 커밋의 내용을 확인함.
52	이력	파일별 작성자	git blame README.md	각 줄을 마지막으로 수정한 커밋/작성자를 확인함.
53	브랜치	브랜치 목록	git branch	로컬 브랜치 목록과 현재 브랜치를 확인함.
54	브랜치	전체 브랜치	git branch -a	로컬과 원격 브랜치를 모두 확인함.
55	브랜치	새 브랜치 생성	git branch feature-login	feature-login 브랜치를 생성함.
56	브랜치	브랜치 이동	git switch feature-login	지정한 브랜치로 이동함.
57	브랜치	생성과 동시에 이동	git switch -c feature-login	새 브랜치를 만들면서 바로 이동함.
58	브랜치	이전 브랜치로 이동	git switch -	직전에 사용하던 브랜치로 돌아감.
59	브랜치	브랜치 이름 변경	git branch -m new-name	현재 브랜치 이름을 new-name으로 변경함.
60	브랜치	main으로 이름 변경	git branch -M main	현재 브랜치를 강제로 main으로 이름 변경함.
61	브랜치	병합된 브랜치 삭제	git branch -d feature-login	병합이 끝난 로컬 브랜치를 안전하게 삭제함.
62	브랜치	브랜치 강제 삭제	git branch -D feature-login	병합 여부와 관계없이 로컬 브랜치를 삭제함.
63	병합	브랜치 병합	git merge feature-login	현재 브랜치에 feature-login 내용을 합침.
64	병합	병합 취소	git merge --abort	충돌 등으로 진행 중인 merge를 취소함.
65	병합	fast-forward 없이 병합	git merge --no-ff feature-login	병합 커밋을 명시적으로 남김.
66	원격	원격 저장소 확인	git remote -v	연결된 GitHub 주소를 확인함.
67	원격	origin 추가	git remote add origin https://github.com/OWNER/REPO.git	현재 로컬 저장소와 GitHub 저장소를 연결함.
68	원격	origin URL 변경	git remote set-url origin https://github.com/OWNER/NEWREPO.git	연결된 GitHub 저장소 주소를 변경함.
69	원격	원격 이름 변경	git remote rename origin github	origin이라는 이름을 github로 변경함.
70	원격	원격 연결 제거	git remote remove origin	로컬 저장소에서 origin 연결 정보만 삭제함.

#	분류	기능	명령어	초보자 설명
71	원격	원격 상세정보	git remote show origin	origin의 브랜치·추적 관계 등을 확인함.
72	업로드	main 최초 push	git push -u origin main	main을 GitHub에 올리고 upstream을 연결함.
73	업로드	현재 브랜치 push	git push	upstream이 설정된 현재 브랜치를 GitHub에 올림.
74	업로드	브랜치 지정 push	git push origin feature-login	특정 브랜치를 GitHub에 업로드함.
75	업로드	모든 태그 push	git push origin --tags	로컬 태그를 GitHub에 모두 업로드함.
76	업로드	원격 브랜치 삭제	git push origin --delete feature-login	GitHub의 특정 원격 브랜치를 삭제함.
77	다운로드	원격 정보 가져오기	git fetch origin	원격 변경사항만 받아오고 현재 파일에는 합치지 않음.
78	다운로드	모든 원격 fetch	git fetch --all	등록된 모든 원격 저장소의 변경정보를 가져옴.
79	다운로드	원격 삭제 브랜치 정리	git fetch --prune	원격에서 삭제된 브랜치의 로컬 참조를 정리함.
80	다운로드	pull	git pull	원격 변경을 받아 현재 브랜치에 합침.
81	다운로드	main pull	git pull origin main	origin의 main 브랜치를 받아 현재 브랜치에 합침.
82	다운로드	rebase 방식 pull	git pull --rebase	불필요한 merge commit을 줄이며 원격 변경을 반영함.
83	되돌리기	커밋 취소 커밋 생성	git revert HEAD	최근 커밋을 반대로 적용하는 새 커밋을 생성함.
84	되돌리기	특정 커밋 revert	git revert abc1234	특정 커밋을 취소하는 새 커밋을 만듦.
85	되돌리기	최근 커밋만 취소, 파일 유지	git reset --soft HEAD~1	커밋만 한 단계 되돌리고 변경은 스테이징 상태로 유지함.
86	되돌리기	최근 커밋 취소, 변경 유지	git reset HEAD~1	커밋을 한 단계 되돌리고 변경 파일은 작업 영역에 유지함.
87	되돌리기	강제 되돌리기	git reset --hard HEAD~1	커밋과 작업 변경까지 제거하므로 매우 주의해서 사용함.
88	임시보관	변경 임시 저장	git stash	커밋 전 변경을 잠시 치워 깨끗한 작업 상태를 만듦.
89	임시보관	메시지와 함께 stash	git stash push -m "작업중"	임시 저장 내용에 설명을 붙임.
90	임시보관	stash 목록	git stash list	임시 저장된 작업 목록을 확인함.
91	임시보관	최근 stash 적용	git stash apply	최근 임시 변경을 적용하되 stash 기록은 남김.
92	임시보관	최근 stash 적용+삭제	git stash pop	최근 임시 변경을 적용하고 stash 목록에서 제거함.
93	임시보관	stash 삭제	git stash drop stash@{0}	특정 임시 저장 기록을 삭제함.

#	분류	기능	명령어	초보자 설명
94	태그	태그 생성	git tag v1.0.0	현재 커밋에 버전 태그를 붙임.
95	태그	태그 목록	git tag	생성된 태그를 확인함.
96	태그	설명 태그	git tag -a v1.0.0 -m "첫 정식버전"	설명과 작성자 정보가 있는 annotated tag를 생성함.
97	GitHub CLI	GitHub 저장소 생성	gh repo create	대화형으로 새 GitHub 저장소를 생성함.
98	GitHub CLI	현재 프로젝트 GitHub에 생성+push	gh repo create my-project --source=. --remote=origin --push	현재 로컬 프로젝트로 GitHub 저장소를 만들고 바로 업로드함.
99	GitHub CLI	저장소 목록	gh repo list	내 GitHub 저장소 목록을 확인함.
100	GitHub CLI	저장소 보기	gh repo view	현재 저장소 정보를 터미널에서 확인함.

5. restore / revert / reset 차이

명령	주 용도	기록 영향	초보자 주의
git restore	파일 변경 복구	브랜치 커밋 기록은 이동하지 않음	수정한 파일을 버릴 때 사용
git revert	과거 커밋 취소	취소용 새 커밋 생성	공유된 GitHub 브랜치에서 비교적 안전
git reset	커밋 위치 이동	커밋 기록에 영향을 줄 수 있음	--hard는 작업 내용까지 삭제 가능

6. .gitignore 기본 예제

Spring Boot / IntelliJ 프로젝트라면 빌드 결과, IDE 설정, 비밀 환경파일 등을 GitHub에 올리지 않도록 .gitignore를 사용함.

```text
build/
target/
.gradle/
.idea/
*.iml
.env
application-secret.yml
*.log
```

7. 실수 방지 핵심 규칙

- git status를 자주 실행하여 현재 상태를 확인함.
- git add . 전에 비밀번호, API Key, .env 같은 비밀 파일이 포함되지 않았는지 확인함.
- 작업 단위가 끝날 때마다 의미 있는 메시지로 commit함.
- 공동 작업에서는 push 전에 pull/fetch로 원격 변경을 확인함.
- git reset --hard와 강제 push는 초보 단계에서는 특별한 이유가 없으면 피함.
- GitHub 인증에는 gh auth login, SSH, Personal Access Token 또는 자격증명 관리자를 사용함.

8. 참고 기준

GitHub 공식 문서의 Git 사용자 설정, GitHub CLI 인증, 원격 저장소 관리, push 절차와 Git 공식 문서의 reset/restore/revert 구분을 기준으로 구성함. 2026년 9월 기준.